

GCC Installation and C Program Execution Guide

Windows • Linux • macOS

Purpose

A beginner-friendly guide that starts with installing a GCC-based C development environment and ends with compiling and running a working C program.

At the end of this guide, you should be able to:

- install a suitable GCC toolchain on Windows, Linux, or macOS;
- verify that the compiler is available from the terminal;
- create a simple C source file;
- compile the source file into an executable program;
- run the executable and understand common compiler errors and warnings.

Recommended for introductory C programming and Compiler Construction laboratory setup.

Contents

1. GCC and the C workflow
2. First C program
3. Windows: install GCC with MSYS2
4. Linux: install GCC
5. macOS: install GNU GCC with Homebrew
6. Understanding the compile command
7. Warnings and C standards
8. Common problems and fixes
9. Quick reference
10. Student verification exercise

Important macOS note

On macOS, the command `gcc` may invoke Apple Clang instead of GNU GCC. Homebrew installs GNU GCC with a versioned executable such as `gcc-16`. The exact number changes when GCC is upgraded, so always check `brew info gcc`.

1. GCC and the C Workflow

GCC stands for GNU Compiler Collection. GCC supports several programming languages. For this guide, we use its C compiler to translate a `.c` source file into an executable program.

```
C source file → GCC compiler → executable program → run the program
```

The operating-system setup is different, but the basic C workflow remains the same.

2. First C Program

We will use the same program on all three systems. Create a file named `hello.c` and enter the following code:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

What this program does:

- `#include <stdio.h>` gives access to standard input/output functions such as `printf()`.
- `main()` is the starting function of the program.
- `printf()` displays text on the terminal.
- `return 0;` reports successful program termination to the operating system.

File-name warning

Make sure the file is really named `hello.c`, not `hello.c.txt`. This is a common problem on Windows.

3. Windows - Install GCC with MSYS2

Windows does not include GNU GCC by default. A practical classroom setup is MSYS2 with the UCRT64 MinGW-w64 GCC toolchain.

Step 1 - Download and install MSYS2

Open the official MSYS2 website and install MSYS2. The usual installation directory is:

```
https://www.msys2.org/  
  
Default installation directory:  
C:\msys64
```

After installation, open MSYS2 UCRT64. Use the UCRT64 terminal for the commands in this guide.

Step 2 - Update MSYS2

```
pacman -Syu
```

If the terminal asks you to close it, close it, reopen MSYS2 UCRT64, and run the same command again if required.

Step 3 - Install GCC

```
pacman -S mingw-w64-ucrt-x86_64-gcc
```

Accept the proposed installation when prompted.

Step 4 - Verify the installation

```
gcc --version
```

If GCC is installed correctly, the terminal prints GCC version information. The exact version number is not important for this test.

Windows - Create, Compile, and Run

Create a folder such as:

```
C:\c-programs
```

Save the earlier program as `hello.c` in that folder. In the MSYS2 UCRT64 terminal, move to the folder:

```
cd /c/c-programs  
ls
```

The `ls` command should show `hello.c`. Compile it:

```
gcc hello.c -o hello.exe
```

If the compilation succeeds, GCC usually prints nothing. Check the directory again:

```
ls
```

You should now see both `hello.c` and `hello.exe`. Run the executable:

```
./hello.exe
```

Expected output:

```
Hello, World!
```

Windows workflow

```
gcc hello.c -o hello.exe  
./hello.exe
```

Optional: use GCC from PowerShell or Command Prompt

To make GCC available outside MSYS2, add this directory to the Windows PATH:

```
C:\msys64\ucrt64\bin
```

After changing PATH, close and reopen the terminal. In PowerShell you can compile with `gcc hello.c -o hello.exe` and run with `.\hello.exe`.

4. Linux - Install GCC

Linux distributions normally provide GCC through their package managers. Use the commands for your distribution.

Distribution	Install command
Ubuntu / Debian	<code>sudo apt update</code> <code>sudo apt install build-essential</code>
Fedora	<code>sudo dnf install gcc</code>
Arch Linux	<code>sudo pacman -S gcc</code>

On Ubuntu/Debian, `build-essential` is recommended because it installs GCC together with common build tools.

Verify GCC

```
gcc --version
```

Create a working directory

```
mkdir c-programs  
cd c-programs
```

Create the file with your preferred editor. For example:

```
nano hello.c
```

Enter the sample C program, save it, and verify that the file exists:

```
ls
```

Linux - Compile and Run

Compile the program:

```
gcc hello.c -o hello
```

If there are no compilation errors, check the directory:

```
ls
```

You should now have the source file and an executable named hello. Run it:

```
./hello
```

Expected output:

```
Hello, World!
```

Linux workflow

```
gcc hello.c -o hello  
./hello
```

Why do we type ./hello?

The ./ means “run the file named hello from the current directory.” Linux usually does not search the current directory automatically when you type a command name.

5. macOS - Install GNU GCC with Homebrew

macOS includes Apple development tools, but the command named `gcc` commonly points to Apple Clang. If the course specifically requires GNU GCC, install it with Homebrew.

Step 1 - Install Apple Command Line Tools

```
xcode-select --install
```

Follow the macOS installation prompts.

Step 2 - Check Homebrew

```
brew --version
```

If Homebrew is not installed, use the official instructions at:

```
https://brew.sh/
```

Step 3 - Install GNU GCC

```
brew install gcc
```

Step 4 - Find the versioned GCC command

```
brew info gcc
```

Homebrew normally installs GNU GCC using a versioned command such as `gcc-16`. The exact suffix changes with future releases.

Do not assume plain `gcc` is GNU GCC

Run `gcc --version`. If the output says Apple clang, that command is Apple Clang. Use the Homebrew versioned GNU GCC command shown by `brew info gcc`.

macOS - Compile and Run

Create a working directory:

```
mkdir c-programs  
cd c-programs
```

Create `hello.c` with an editor, for example:

```
nano hello.c
```

After saving the sample program, compile it using your installed versioned GNU GCC command.
Example:

```
gcc-16 hello.c -o hello
```

Run the executable:

```
./hello
```

Expected output:

```
Hello, World!
```

macOS GNU GCC workflow

```
gcc-<version> hello.c -o hello  
./hello
```

Use `brew info gcc` to determine the current versioned command.

6. Understanding the GCC Compile Command

Consider the command:

```
gcc program.c -o program
```

Part	Meaning
gcc	Run the GNU C compiler.
program.c	The input C source file.
-o	Choose the name of the output file.
program	The executable program that GCC should create.

If you omit `-o` on Linux/macOS, GCC normally creates an executable named `a.out`:

```
gcc program.c  
./a.out
```

Giving the executable a meaningful name is easier to understand:

```
gcc program.c -o program  
./program
```

On Windows, it is conventional to use the `.exe` suffix:

```
gcc program.c -o program.exe  
./program.exe
```

7. Warnings and C Standards

For learning C, compile with warning options enabled. They help detect suspicious code even when the program can still be compiled.

```
gcc -Wall -Wextra hello.c -o hello
```

-Wall enables many common warnings, while -Wextra enables additional warnings.

Selecting a C language standard

You can tell GCC which C standard to use:

```
gcc -std=c99  program.c -o program
gcc -std=c11  program.c -o program
gcc -std=c17  program.c -o program
gcc -std=c23  program.c -o program
```

A useful classroom command for a modern compiler is:

```
gcc -std=c23 -Wall -Wextra program.c -o program
```

Compatibility note

C23 support depends on the GCC version. If a machine has an older GCC release and rejects -std=c23, use the standard required by your course or upgrade the compiler.

Compiler error vs. compiler warning

Message type	Meaning
Error	The compiler cannot successfully build the program until the error is fixed. Example: a missing semicolon that breaks the syntax.
Warning	GCC found code that may be incorrect or risky. The executable may still be produced, but the warning should be investigated.

8. Common Problems and Fixes

gcc: command not found

GCC is not installed, the terminal has not been restarted, or GCC is not on PATH. Verify the installation command for your operating system.

'gcc' is not recognized on Windows

Open MSYS2 UCRT64 and verify GCC there. If you want PowerShell/Command Prompt support, add C:\msys64\ucrt64\bin to PATH and reopen the terminal.

No such file or directory

You are probably in the wrong working directory or the file name is different. Run `pwd` (Linux/macOS/MSYS2) and `ls` to inspect the current directory.

./hello: No such file or directory

The executable probably was not produced. Re-run the GCC compile command and read any errors.

gcc --version says Apple clang on macOS

This is normal for the system `gcc` command. Use `brew info gcc` to find the Homebrew GNU GCC versioned executable.

The program does not recompile after editing

Save the source file, then run the GCC command again. GCC does not automatically rebuild your program when the source changes.

Example syntax error

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n")    /* missing semicolon */
    return 0;
}
```

GCC will report an error. Add the missing semicolon and compile again.

9. Quick Reference

System	Install / verify	Compile	Run
Windows	MSYS2 UCRT64 gcc --version	gcc hello.c -o hello.exe	./hello.exe
Linux	Install with distro package manager gcc --version	gcc hello.c -o hello	./hello
macOS	brew install gcc brew info gcc	gcc-<version> hello.c -o hello	./hello

Recommended compile commands

Windows:

```
gcc -Wall -Wextra program.c -o program.exe
```

Linux:

```
gcc -Wall -Wextra program.c -o program
```

macOS GNU GCC:

```
gcc-<version> -Wall -Wextra program.c -o program
```

The essential idea to remember

1. Write source.c
2. Compile source.c
3. GCC creates an executable
4. Run the executable
5. Observe the output

10. Student Verification Exercise

Use this exercise to confirm that your system is ready for C programming.

Task

1. Verify that your compiler command works and record the compiler version.
2. Create a file named `student.c`.
3. Write a program that prints your name and the line `GCC is working!`.
4. Compile the program with `-Wall -Wextra`.
5. Run the executable.
6. Change the printed message, save the file, recompile, and run it again.

Example source

```
#include <stdio.h>

int main(void)
{
    printf("Your Name\n");
    printf("GCC is working!\n");
    return 0;
}
```

Verification checklist

- The compiler version command works.
- The source file uses the `.c` extension.
- The program compiles without errors.
- The executable file is created.
- The program runs and displays the expected output.
- After modifying the source, recompiling changes the executable output.

You are ready

If you can complete all six checks, your GCC environment is ready for introductory C programming.

Official setup references

- MSYS2: <https://www.msys2.org/>
- Homebrew: <https://brew.sh/>
- Homebrew GCC formula information: <https://formulae.brew.sh/formula/gcc>